Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Veritabanı Yönetim Sistemi (VYS) tanımı ve sistem karmaşıklığı
  - Çok kullanıcılı, eşzamanlı, güvenli servis sağlayan yapı
  - Disk gibi güvenilirliği düşük bir ortamda binlerce kullanıcının verisinin tutulması
  - VYS'nin düşük seviyeli dillerle (C vb.) yazılması; sistem seviyesinde yazılım gerektirmesi
- VYS dosya yapısı (4 temel dosya)
  - Veri dosyaları: Verinin kendine has formatta, sıkıştırılmış şekilde saklanması (program-data independence)
  - Index dosyaları: Veri dosyalarına hızlı erişim sağlayan küçük boyutlu veri yapıları; veri tekrar (replication) içerir ama özet/çekirdek niteliğindedir
  - Log dosyaları: Kurtarma (recovery) ve veri bütünlüğü (data integrity) için; güvenlik ve anomali tespiti de sağlanır
  - Veri sözlüğü (metadata): Şema bilgileri, dosya büyüklükleri, niteliklerin değer dağılımları gibi istatistik bilgileri; sorgu optimizasyonu için kritik
- Transaction kavramı ve ACID kriterleri
  - Birden fazla VYS eylemi içeren program parçaları
  - Atomiklik, Tutarlılık (Consistency), İzolasyon, Dayanıklılık (Durability)
- Veri modeli ve tasarım aşamaları
  - Kavramsal tasarım → Mantıksal modelleme → Fiziksel tasarım
  - Geri beslemeli yapı; ancak ilk adımların sağlam atılması gerektiği
  - Kavramsal tasarımda ER ve UML notasyonları kullanılır; ders içinde ER notasyonu ağırlıklı
- Şema ve Instance kavramları
  - Şema: Veritabanının yapısı, tablo isimleri, nitelikler, veri tipleri, kısıtlamalar, fonksiyonlar, triggerlar (description)
  - Instance (Database state): Belirli bir andaki veritabanında saklanan gerçek verilerin durumu (snapshot)
- İlişkisel modelin tarihsel gelişimi ve gücü
  - 1970'lerden günümüze ~50 yıllık köklü standart
  - Hem basit hem sağlam (salabet) yapısı
  - SQL standardı 2016'ya kadar 8 revizyon geçirmiş
  - MySQL, PostgreSQL, Oracle gibi sistemler ilişkisel model esaslı
- Veri modeli alternatifleri
  - Object-oriented modeller: Programlama dillerinden kökenli, nesne yönelimli kavramları VYS'ye taşımaya yönelik; çok tutmamış ama NoSQL'e öncülük etmiş
  - Object-relation modeller: İlişkisel modele nesne yönelimli kavramları (inheritance, özel veri tipleri, object ID) entegre etme; SQL 99'dan itibaren standartlara girmiş
  - Hierarchical modeller: XML gibi iç içe, öz-yinelemeli yapılar; şema veri ile birlikte, esnek
  - NoSQL: Şemasız ortam, esnek yapı; ilişkisel, nesne yönelimli ve nesne-ilşki modellerini çatısı altında toplayan geniş bir yaklaşım
- VYS kullanıcıları ve roller
  - DBA (Veritabanı Yöneticisi): Tasarım, işletim, güvenlik, erişim yetkileri, donanım/yazılım kaynak yönetimi, RAID yapılandırması
  - Tasarımcı: Verinin modellenmesi, gerçekleme, kullanıcılarla gereksinim analizi

  - Sistem Analisti: Son kullanıcı gereksinimlerinin listelenmesi
  - Uygulama Yazılımcısı: VYS'ye erişim yapacak uygulamaların tasarlanması ve gerçekleştirilmesi
  - Son Kullanıcı: Farklı bilgisayar yakınlığı seviyelerinde; görsel arayüz, browser veya konsol üzerinden erişim
  - Sistem Yazılımcısı: VYS'nin kendisini yazan grup (C/C++ gibi düşük seviyeli dillerle)
- DDL ve DML
  - Data Definition Language: CREATE TABLE, DROP TABLE, CREATE SCHEMA gibi şema tanımlama/değiştirme komutları
  - Data Manipulation Language: SELECT, INSERT, UPDATE, DELETE; veri sorgulama ve değiştirme
  - SQL her iki imkanı da sağlar
- Programlama dilleri ile VYS iletişimi
  - Komut konsolu: Siyah-beyaz terminal arayüzü; connection açma/kapama, şema tanımlama, sorgu gönderme
  - Gömülü SQL (Embedded SQL): Java'da JDBC gibi; uygulama dilinin içinde SQL
  - Procedure Call: API üzerinden fonksiyon çağrısı (JDBC, OCI vb.)
  - Stored Procedure: VYS'nin kendi sağladığı ortamda prosedür yazma (PL/SQL, PL/pgSQL)
  - IDE'ler: Kapsamlı uygulama geliştirme ortamları
- VYS Utility'leri (sistem araçları)
  - Bulk loading (toplu yükleme): CSV, Excel gibi formatlardan otomatik dönüşüm ve yükleme
  - Backup: Periodik olarak tape veya mirror disk'e yedekleme
  - Reorganization: Dosya yapılarının yeniden düzenlenmesi; adaptif/otomatik reorganizasyon akademik araştırma konusu
  - Performance monitoring: Darboğaz tespiti, kaynak planlaması
  - Raporlama: Verilerin grafiksel olarak sunumu
- Dosya işleme vs VYS karşılaştırması
  - Dosya işlemede metadata programın içinde; format değişikliği programı kırar
  - Random access yok; satır satır okuma (BufferedReader)
  - VYS'de metadata bağımsız, indeks yapıları mevcut, çok az kodla çok iş yapılabiliyor
  - Program-veri bağımsızlığı: Verinin saklanma şekli değişse de program etkilenmez
- VYS ne zaman kullanılmaz
  - Maliyet: Donanım, lisans, uzman personel gereksinimi
  - Basit, iyi tanımlanmış, kolay değişmeyen uygulamalar
  - Gerçek zamanlı (ana hafıza esaslı) işlem gereksinimleri
  - Eşzamanlı kullanımın olmadığı ortamlar
  - Sorgulamanın kritik olmadığı, sadece sıralı erişim yapılan durumlar
  - VYS modelleme sınırlamalarının karmaşık yapıyı tam yansıtamaması
- Üç şema mimarisi
  - Internal (fiziksel) şema: Verinin nasıl saklandığı, indeks yapıları, erişim metotları
  - Conceptual (kavramsal) şema: Veritabanının bütün yapısı, tutarlılık tanımı; ilişkisel model, document model vb.
  - External (dış) şema: Farklı kullanıcı profillerinin veriyi görme biçimi; view'lar
- Veri bağımsızlığı (Data Independence)
  - Physical Data Independence: Internal şemadaki değişikliklerden (örn. indeks değişimi) üst katmanlar etkilenmez
  - Logical Data Independence: Conceptual şemadaki değişikliklerden (örn. tablo bölme) external view'lar etkilenmez; fonksiyon isimleri aynı kalır, içleri değişir
- VYS kullanım mimarileri
  - Merkezi (Centralized): Tek makinede VYS + kullanıcılar
  - İki katmanlı Client-Server: Database server + client'lar; connection kapasitesi sınırlı
  - Üç katmanlı mimari: GUI/Web → Application Server → Database Server; connection pooling ile sınırlı connection sayısının çok kullanıcıya dağıtılması
- VYS sınıflandırması
  - Tek/çok kullanıcılı
  - Merkezi / dağıtık (distributed)
  - Homojen / heterojen
  - Uygulama alanları: Multimedia, spatial/temporal, information retrieval, data mining,

object-relational
- Veritabanı tasarım süreci
  - Gereksinim toplama ve analiz → Kavramsal tasarım (ER diyagramı) → Mantıksal tasarım (ilişkisel model tabloları) → Normalizasyon → Fiziksel tasarım (indeks seçimi) → Gerçekleme
  - View integration: Farklı kullanıcı view'larının (customer view, sales view vb.) birleştirilerek kapsamlı ER diyagramı oluşturulması
  - Normalizasyon ile doğruluk artırılır; denormalizasyon ile doğruluktan ödün verip performans artırılabilir (trade-off)
  - Geri besleme ile sistem zamanla iyileştirilebilir
- ER Modeline giriş - Temel kavramlar
  - Varlık (Entity): Diğerlerinden ayırt edilebilen her nesne
  - Varlık kümesi (Entity set): Benzer varlıkların oluşturduğu küme
  - Nitelik (Attribute): Varlığı tanımlayan değerler; her niteliğin domain'i (veri tipi + değer aralığı) var
  - Bağıntı (Relationship): İki veya daha çok varlık arasındaki olayı tanımlayan kavram
  - Bağıntı kümesi: Aynı türdeki bağıntıların kümesi
- ER notasyonu
  - Varlık setleri: Dörtgen (dikdörtgen)
  - Nitelikler: Yuvarlak (oval) içinde nitelik ismi
  - Bağıntılar: Eşkenar dörtgen; varlık setlerini bağlar
  - Anahtar (Key): Altı çizili nitelik → biricik (unique); varlık kümesindeki her varlık için farklı
  - Kompozit nitelik: Alt niteliklere ayrılan nitelik (örn. kayıt numarası = eyalet + numara)
  - Çok değerli (Multi-valued) nitelik: Çift çizgili yuvarlak; bir varlığın birden çok değeri olabilir (örn. renk, lokasyon)
  - Bağıntı derecesi: İkili bağıntı (degree 2), üçlü bağıntı vb.
  - Bağıntı kardinalitesi: Bire-bir, bire-n, n'ye-n

Hocanın Özellikle Vurguladığı Kısımlar

- "Log deyince kurtarma aklınıza gelmesi lazım"
  - İlk sırada veri kurtarma ve doğruluk (data integrity); güvenlik ikinci sırada
  - Hoca eski ders notlarında ilk sıraya güvenlik yazdığını, bunun yanlış olduğunu düzeltti
- "Index deyince hız aklınıza gelmesi lazım"
  - Index dosyaları veri dosyalarına göre çok küçük (%1 veya daha az) ama olmazsa olmaz
  - Index olmadan veri erişim hızı aşırı düşer
- "Veri sözlüğü NoSQL veritabanlarında olmazsa olur, hatta olmazsa daha iyi olur"
  - NoSQL şemasız yaklaşımı; esnek yapı; ancak ilişkisel modelde metadata kritik
- "Disk ana hafıza arasındaki fark sistemin karmaşıklığını artıran en önemli etken"
  - Disk ~100.000 kat yavaş ana hafızaya göre (4-5 saniye ile 24 saat arası fark)
  - Tampon (buffer) yönetimi, kurban seçimi algoritmaları gerekli
- "Program-veri bağımsızlığı" kavramının önemi
  - Dosya işlemede metadata programın içinde → VYS'de metadata bağımsız
  - Bir öğrencinin sorusuyla pekiştirildi: VYS araya girerek program ve verinin depolanmasını ayırt ediyor
- "Az kod çok iş yapıyor" prensibi
  - SQL ile bir cümleyle yapılan iş, dosya işlemede onlarca satır kod gerektiriyor
  - Katalog bilgisi, indeks, random access VYS tarafından otomatik sağlanıyor
- "VYS her zaman kullanılmaz" uyarısı
  - Maliyet, basit uygulamalar, gerçek zamanlı gereksinimler, eşzamanlı kullanım yoksa gereksiz yük
- "Tasarım sürecinde ilk adımların sağlam atılması"
  - Kavramsal tasarım ve mantıksal düzenleme değiştirilmesi zor; fiziksel tasarımda geri besleme daha kolay
  - Normalizasyon dersin kapsamından çıkarılmış (pratik odaklı), ama bilinmesi gereken bir konu

- ER notasyonunda detaylar:
  - Anahtar (altı çizili): Biriciklik; birden çok anahtar olabilir, ilişkisel modele geçerken biri primary key seçilir
  - Kompozit anahtar: Birden çok niteliğin birleşimi ancak unik olabilir (örn. eyalet + numara)
  - Multi-valued nitelik: Çift çizgi; departmanın birden çok lokasyonu, arabanın birden çok rengi olabilir

Kısa Tekrar Notları

- VYS 4 dosya türü: Veri, Index, Log, Veri Sözlüğü
- ACID: Atomiklik, Consistency, Isolation, Durability
- DDL = şema tanımlama; DML = veri sorgulama/değiştirme
- Şema = yapı tanımı; Instance = o andaki veri durumu
- İlişkisel model 1970'lerden beri; hem basit hem sağlam
- SQL 2016 standardı; 8 revizyon geçmiş
- Object-oriented modeller tutmadı → NoSQL'e öncülük etti
- Object-relation: İlişkisel modele inheritance, özel veri tipleri eklendi (SQL 99+)
- Physical data independence: İç şema değişse üst katmanlar etkilenmez
- Logical data independence: Kavramsal şema değişse dış view'lar etkilenmez
- 3 katmanlı mimari: GUI → Application Server → DB Server; connection pooling
- ER modeli: Varlıklar (dörtgen), Bağıntılar (eşkenar dörtgen), Nitelikler (yuvarlak)
- Anahtar = altı çizili nitelik = biricik
- Kompozit nitelik = alt niteliklere ayrılan nitelik
- Multi-valued nitelik = çift çizgili yuvarlak
- Bağıntı kardinalitesi: 1:1, 1:N, N:M
- Tasarım süreci: Gereksinim → Kavramsal (ER) → Mantıksal (ilişkisel tablo) → Normalizasyon → Fiziksel → Gerçekleme
- Normalizasyon doğruluğu artırır; denormalizasyon performansı artırır (trade-off)

Detaylı Açıklamalar

- **VYS Dosya Yapısı ve Sistem Karmaşıklığı**: Veritabanı yönetim sistemi 4 temel dosya üzerinde çalışır. Veri dosyaları verinin kendine has formatta saklandığı yapılardır; ASCII dosyası gibi açıp bakılabilir bir yapı değildir, sıkıştırılmış ve özel düzenlidir. Index dosyaları veri dosyalarına hızlı erişim sağlayan, veri dosyalarının çok küçük bir yüzdesi kadar yer kaplayan ama veri tekrarı (replication) içeren yapılardır; ağacın çekirdeği gibi düşünülebilir. Log dosyaları verinin kurtarılması ve doğruluğunun garanti edilmesi için kullanılır; güvenlik ve anomali tespiti de sağlar. Veri sözlüğü (metadata) ise tüm dosyaların şema bilgilerini, büyüklüklerini ve niteliklerin değer dağılımları gibi istatistik bilgilerini tutar; bu istatistikler sorgu optimizasyonu için kritik öneme sahiptir. Optimizasyon sistemin en karmaşık modülüdür.
- **Transaction ve ACID**: Transaction, birden fazla veritabanı eylemi içeren program parçalarıdır. ACID kriterleri (Atomiklik, Consistency, Isolation, Durability) bir transaction'ın sağlaması gereken özelliklerdir. Elektrik kesilmesi gibi durumlarda yarıda kalan işlemlerin geri sarılması (rollback) veya tamamlanan işlemlerin doğruluğunun garanti edilmesi (commit) sistemin sorumluluğundadır. Eşzamanlılık kontrolünde kilit mekanizmaları kullanılır; aynı kayda birden çok kişinin yazması engellenir.
- **Üç Şema Mimarisi ve Veri Bağımsızlığı**: VYS üç katmanlı bir soyutlama ile çalışır. En altta internal (fiziksel) şema verinin nasıl saklandığını tanımlar (indeks yapıları, erişim metotları). Ortada conceptual (kavramsal) şema tüm veritabanının yapısını ve tutarlılığını tanımlar (ilişkisel model tabloları). En üstte external (dış) şema farklı kullanıcı profillerinin veriyi görme biçimini tanımlar. Physical data independence sayesinde internal şemadaki değişiklikler (örn. B+ tree yerine yeni bir indeks yapısı) üst katmanları etkilemez. Logical data independence sayesinde conceptual şemadaki değişiklikler (örn. öğrenci tablosunu aktif öğrenci ve mezun olarak ikiye bölme) external view'ları etkilemez; fonksiyon isimleri aynı kalır, yalnızca iç implementasyonları değişir.

- **VYS Kullanım Mimarileri**: Merkezi mimaride tüm süreçler tek makinededir; connection kapasitesi sınırlıdır. İki katmanlı client-server mimarisinde database server ayrı bir makinededir. Üç katmanlı mimaride araya application server eklenir; bu katman business logic'i çalıştırır ve connection pooling ile sınırlı sayıdaki veritabanı bağlantısını milyonlarca kullanıcıya dağıtır. Connection pooling, buffer pool mantığına benzer şekilde çalışır; bağlantılar scheduled edilir ve kurban seçimi yapılır.
- **ER Modeli Temel Kavramları**: Varlık (entity) diğerlerinden ayırt edilebilen her nesnedir. Benzer varlıklar varlık kümesi (entity set) oluşturur. Nitelikler (attributes) varlığı tanımlayan değerlerdir; her niteliğin domain'i hem veri tipini hem değer aralığını kapsar. Bağıntı (relationship) iki veya daha çok varlık arasındaki ilişkiyi tanımlar. ER diyagramında varlık setleri dörtgen, nitelikler yuvarlak, bağıntılar eşkenar dörtgen ile gösterilir. Anahtar (key) altı çizili nitelik olarak gösterilir ve varlık kümesindeki her varlığı biricik kılar. Birden çok anahtar olabilir; ilişkisel modele geçişte biri primary key olarak seçilir. Kompozit nitelikler alt niteliklere ayrılabilir (örn. kayıt numarası = eyalet + numara; tek başına ne eyalet ne numara unik olabilir ama birlikte unik olurlar). Multi-valued nitelikler çift çizgili yuvarlakla gösterilir ve bir varlığın birden çok değeri alabilir (örn. departmanın birden çok lokasyonu).
- **Veritabanı Tasarım Süreci**: Tasarım süreci gereksinimlerin toplanması ve analizi ile başlar. VYS tasarımcısı müstakbel kullanıcılarla görüşerek veri ve işlevsel gereksinimleri belgeler. Kavramsal tasarımda ER diyagramı ile küçük dünya modellenir; farklı kullanıcı view'ları (customer view, sales view vb.) oluşturulup view integration ile birleştirilir. Mantıksal tasarımda ER diyagramı ilişkisel model tablolarına dönüştürülür. Normalizasyon ile veri bütünlüğü garanti altına alınır (teorik ağırlıklı konu). Fiziksel tasarımda indeks yapıları ve erişim metotları belirlenir; bu aşama VYS'ye özeldir (Oracle, PostgreSQL, SQL Server farklı yapar). Denormalizasyon ile doğruluktan ödün vererek performans artırılabilir. Gerçekleme sonrası geri besleme ile sistem sürekli iyileştirilir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.